In C++, you can directly create and use a **Linked List using STL (`std::list`)** without defining a `Node` class, `head`, `tail`, or `next` pointer.

Let's understand everything from creating a linked list to performing all the important operations, just like you do with `vector`.

1. Creating a Linked List using STL

First, include the `<list>` header.

```
#include &lt;iostream&gt;
#include &lt;list&gt;
using namespace std;

int main() {
    list&lt;int&gt; ll;                // Empty linked list
    list&lt;int&gt; ll2 = {10, 20, 30}; // Initialized linked list
    list&lt;int&gt; ll3(5, 10);        // 5 nodes with value 10

    return 0;
}
```

The STL `list` is a **doubly linked list**. Each internal node has connections to both the previous and next nodes, but STL manages these pointers automatically.

2. Important Linked List operations in STL

Operation	STL method	Complexity
Insert at front	ll.push_front(10)	O(1)
Insert at back	ll.push_back(20)	O(1)
Delete from front	ll.pop_front()	O(1)
Delete from back	ll.pop_back()	O(1)
Access first element	ll.front()	O(1)
Access last element	ll.back()	O(1)
Get number of elements	ll.size()	O(1)
Check if empty	ll.empty()	O(1)
Reverse list	ll.reverse()	O(n)
Sort list	ll.sort()	O(n log n)
Remove a value	ll.remove(10)	O(n)
Remove adjacent duplicates	ll.unique()	O(n)
Clear all elements	ll.clear()	O(n)
Insert at position	ll.insert(it, val)	O(1)*
Erase an element	ll.erase(it)	O(1)*
Search for value	find(ll.begin(), ll.end(), val)	O(n)

* insert() and erase() are O(1) once the iterator is at the required position. Finding a position by index takes O(n).

3. Complete C++ program: All major operations

This example shows how to use STL directly rather than implementing a custom linked-list class.

```
#include &lt;iostream&gt;
#include &lt;list&gt;
#include &lt;algorithm&gt;
#include &lt;iterator&gt;
using namespace std;

void printLL(const list&lt;int&gt;&amp;amp; ll) {
    for (int x : ll) {
        cout &lt;&lt; x &lt;&lt; " -&gt; ";
    }
    cout &lt;&lt; "NULL\n";
}

int main() {
    // 1. Create Linked List
    list&lt;int&gt; ll = {10, 20, 30};

    cout &lt;&lt; "Original: ";
    printLL(ll);

    // 2. Push Front
    ll.push_front(5);
    printLL(ll); // 5 10 20 30

    // 3. Push Back
    ll.push_back(40);
    printLL(ll); // 5 10 20 30 40

    // 4. Pop Front
    ll.pop_front();
    printLL(ll); // 10 20 30 40

    // 5. Pop Back
    ll.pop_back();
    printLL(ll); // 10 20 30

    // 6. Insert at index 2
    auto it = ll.begin();
    advance(it, 2);
    ll.insert(it, 25);
    printLL(ll); // 10 20 25 30
```

```
    // 7. Delete element at index 1
    it = ll.begin();
    advance(it, 1);
    ll.erase(it);
    printLL(ll); // 10 25 30

    // 8. Search for value
    it = find(ll.begin(), ll.end(), 25);

    if (it != ll.end()) {
        cout &lt;&lt; "25 Found\n";
        cout &lt;&lt; "Index: "
             &lt;&lt; distance(ll.begin(), it) &lt;&lt; endl;
    } else {
        cout &lt;&lt; "Not Found\n";
    }

    // 9. First and Last
    cout &lt;&lt; "Front: " &lt;&lt; ll.front() &lt;&lt; endl;
    cout &lt;&lt; "Back: " &lt;&lt; ll.back() &lt;&lt; endl;

    // 10. Size
    cout &lt;&lt; "Size: " &lt;&lt; ll.size() &lt;&lt; endl;

    // 11. Reverse
    ll.reverse();
    printLL(ll); // 30 25 10

    // 12. Sort
    ll.sort();
    printLL(ll); // 10 25 30

    // 13. Remove a value
    ll.remove(25);
    printLL(ll); // 10 30

    // 14. Check empty
    cout &lt;&lt; boolalpha &lt;&lt; ll.empty() &lt;&lt; endl;

    // 15. Clear Linked List
    ll.clear();
    cout &lt;&lt; "After clear: ";
```

```
    printLL(ll);

    return 0;
}
```

4. Understanding begin(), end(), next(), and prev()

This is the most important part to understand because you previously used `Node* temp` and `temp->next` in your custom linked list.

Consider:

```
list&lt;int&gt; ll = {10, 20, 30, 40};
```



Conceptual doubly linked list. begin() points at the first element; back() accesses the last. end() is a special past-the-end iterator, not the last element.

Here's how you navigate it.

```
auto it = ll.begin();

cout &lt;&lt; *it;          // 10

++it;
cout &lt;&lt; *it;          // 20

cout &lt;&lt; *next(it);    // 30

cout &lt;&lt; *prev(it);    // 10

cout &lt;&lt; *prev(ll.end()); // 40
```

Difference between `next()` and `++it`:

```
auto it = ll.begin();

++it;         // Changes it to point at 20

auto x = next(it); // x points at 30
                   // it still points at 20
```

Note that `next()` and `prev()` return iterators, not the actual node or its value. You use * to obtain the value.

5. How to insert or delete at any index

Unlike vectors, `std::list` does not support indexing such as `ll[2]`.

You need an iterator and `advance()`.

Example: Insert 99 at index 2

Before insertion:



After inserting 99 at index 2:




```
list&lt;int&gt; ll = {10, 20, 30, 40};

auto it = ll.begin();

advance(it, 2);      // Move 2 steps

ll.insert(it, 99);   // Insert before 30
```

Result: `10 -> 20 -> 99 -> 30 -> 40`

To delete an element at index 2:

```
auto it = ll.begin();

advance(it, 2);

ll.erase(it);        // Removes 99
```

Result: `10 -> 20 -> 30 -> 40`

Always validate the index before using `advance()` so you don't move beyond the valid range. `erase()` also requires an iterator pointing to an existing element, not `end()`.

6. What if you specifically want a singly linked list?

Use `forward_list<int>`:

```
#include &lt;forward_list&gt;
using namespace std;

forward_list&lt;int&gt; ll = {10, 20, 30};

ll.push_front(5);
ll.pop_front();

auto it = ll.begin();
++it;                // Move to next element

ll.insert_after(it, 99);
ll.erase_after(it);
```

Unlike `std::list`, `std::forward_list` has no built-in `push_back()`, `pop_back()`, or backward iterator movement.